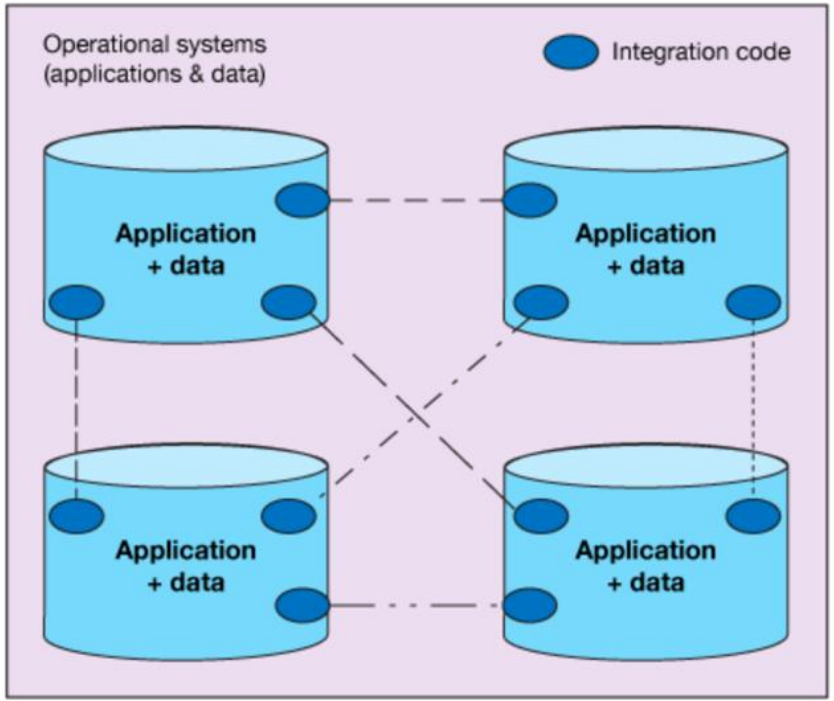
Ingeniería de Software II

Arquitecturas de Integración

Al principio de los 90s…



- Conexión “punto a punto”
- Interfaces específicas para cada sistema contra cada Sistema

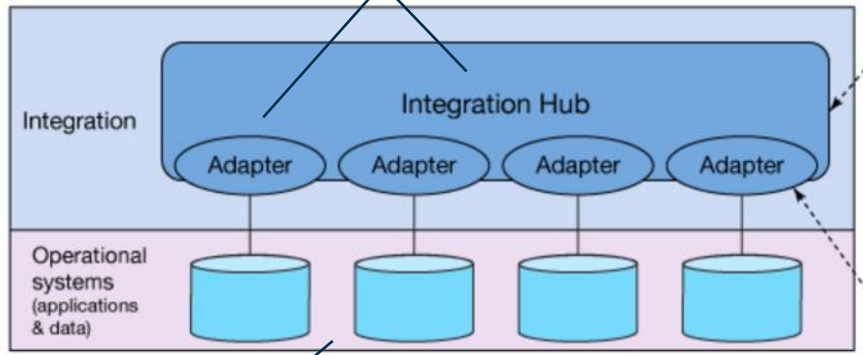
+ Simple
- Desorden

A fines de los 90s…

Evitar la maraña de integraciones punto a punto

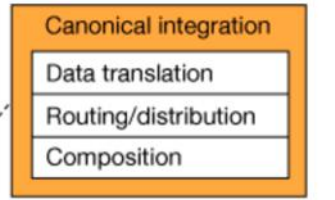
Intermediado que “rutea”: transformación de datos, mediación de protocolos, orquestación simple y desacoplamiento entre aplicaciones

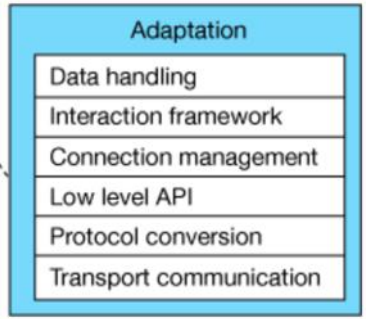
Colas, HTTP/SOAP, RMI, JDBC…



Middleware

Apps



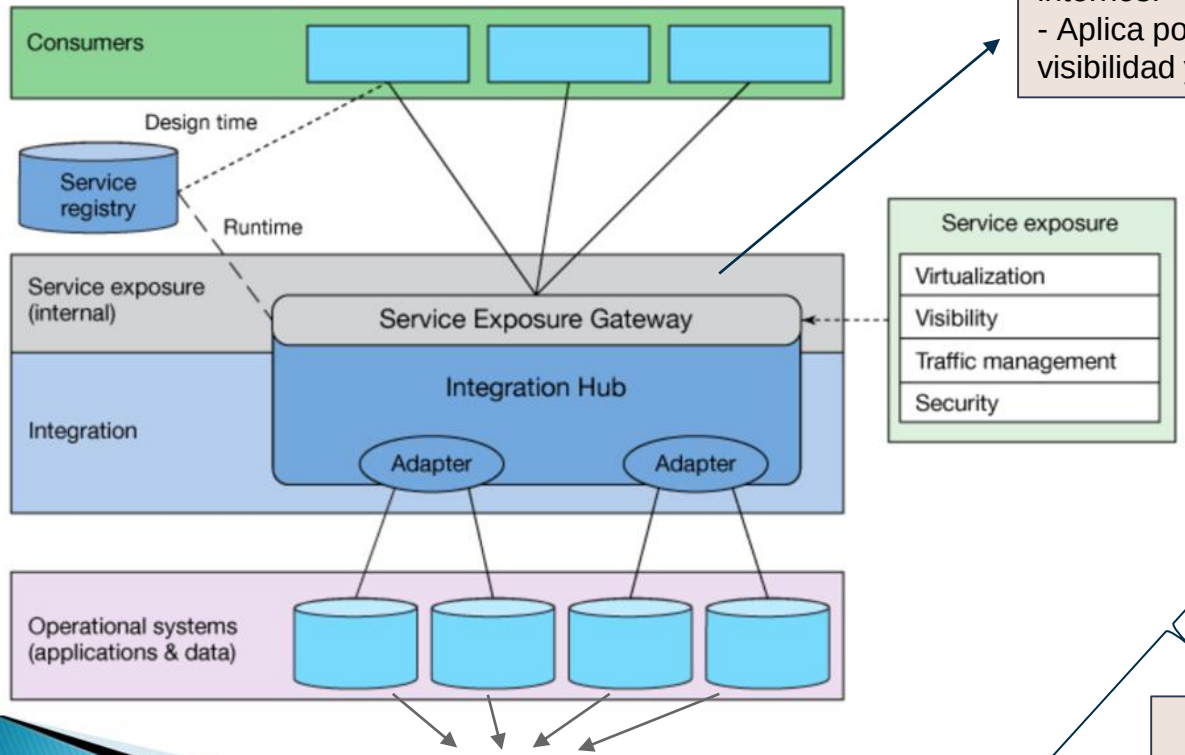


Sistemas independientes

5

A principios del 2000...

Clientes (consumen servicios del ESB)



Nuevo:
- Gestiona la exposición de servicios internos.
- Aplica políticas de seguridad, visibilidad y virtualización.

ESB

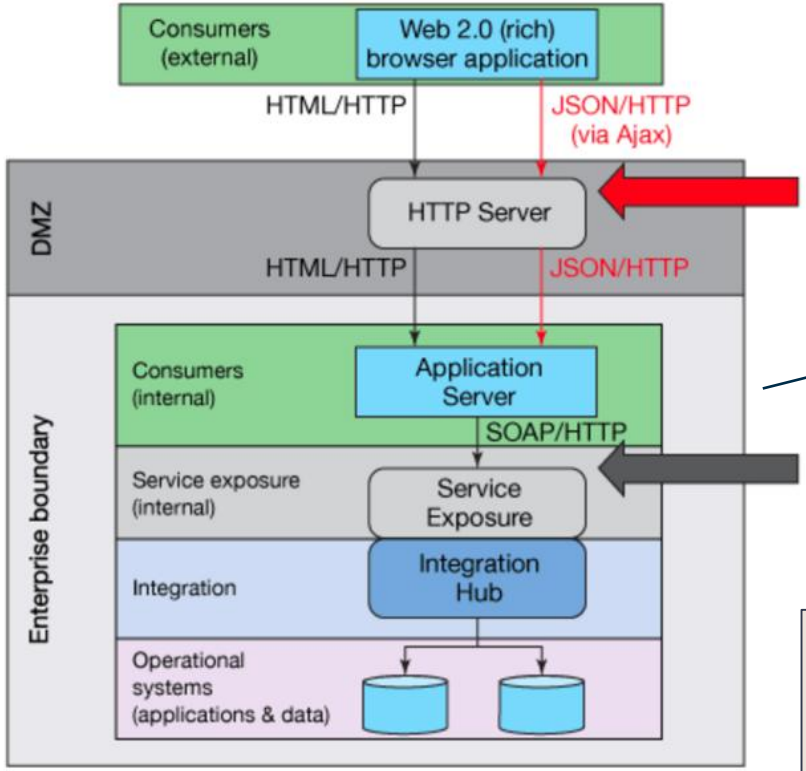
Tendencia EAI:
- Eliminates Data Silos
- Automates Workflows
- Unifies Data

Cobol, app, legacy, ERP, CRM, …

Permite reutilización, bajo acoplamiento y gestión centralizada de servicios

Cerca de 2010...

Contexto: madurez de la Web 2.0, AJAX, JSON y las primeras APIs REST



Fine-grained application scoped "RESTful API" exposure

Servicios como capa de integración interna

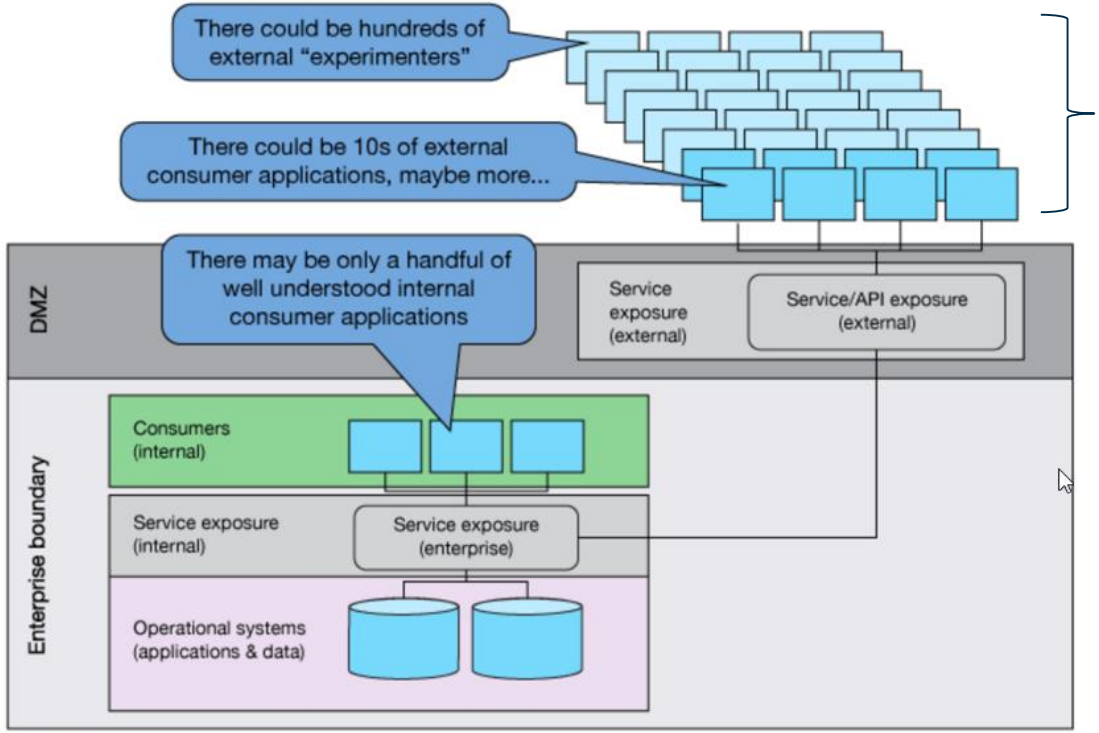
Course-grained enterprise scoped "web service" exposure

SOA

- Apertura hacia el exterior (web y móviles).
- Integraciones más chicas y rápidas, sin tanta infraestructura
- Mayor granularidad: ya no exponer "servicios empresariales enormes" (por ej, ESB), sino funcionalidades granulares específicas.

- Las APIs ya no son solo conectores: son interfaces de negocio.
- La empresa se convierte en una plataforma abierta, capaz de escalar exponencialmente gracias a integraciones externas.
- Se pasa de conectar sistemas a conectar ecosistemas.

Despues del 2010...



Diferencial:
- API como interfaz pública
- Otros construyen valor alrededor
- No es solo un integrador interno

8

Service Oriented Architecture (SOA)

Que es SOA?

- Es un **estilo** arquitectónico
- Las funcionalidades de negocio deben implementarse como **servicios independientes**
- Deben tener las **interfaces bien definidas** y contratos públicos
- Pueden ser **descubiertos**, invocados y reutilizados por otros sistemas dentro (o fuera) de la organización.

*SOA organiza una aplicación (o varias) como un **ecosistema de servicios autónomos** que colaboran entre sí mediante mensajes estandarizados.*

Como si fuera un “catálogo” de servicios donde podés buscar qué hace cada uno, cómo se llama y cómo invocarlo: **Service Registry**

Principios de diseño

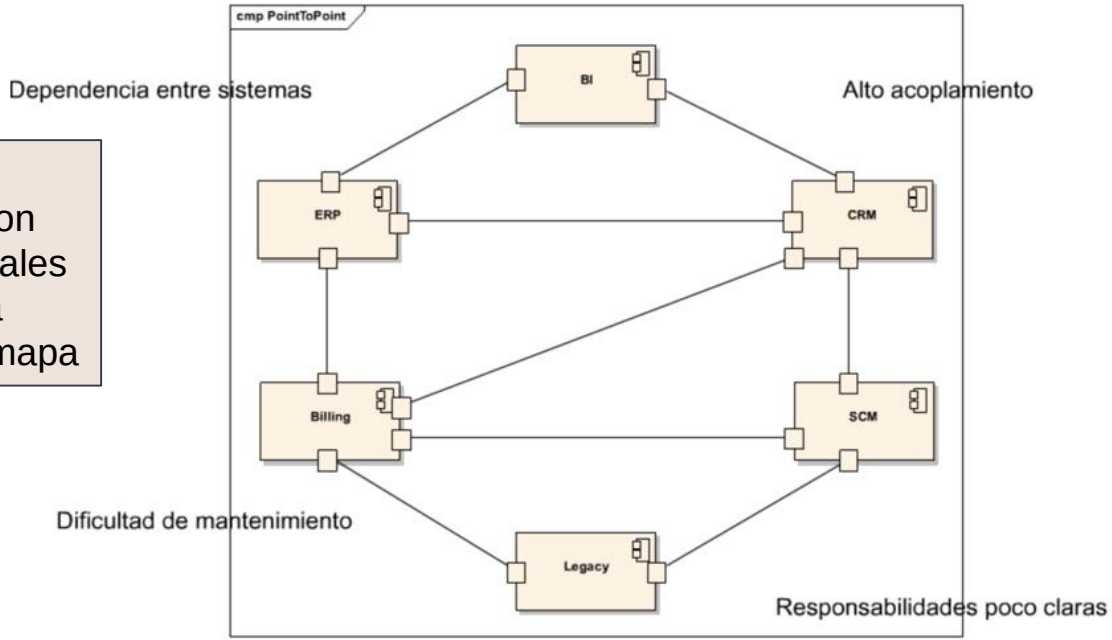
- **Contratos estandarizados:** los servicios dentro del mismo inventario, deben estar acordes a los mismos principios de diseño de contratos.
- **Bajo acoplamiento:** de los servicios respecto a su entorno y, a su vez, imponer bajo acoplamiento a sus consumidores.
- **Abstracción:** con unos contratos que sólo contienen lo esencial y sin que exista más información que la publicada en los contratos.
- **Reutilización:** contenedores de lógica agnóstica que pueda ser fácilmente reutilizada en diferentes contextos.
- **Autonomía:** ejerciendo un alto control sobre el entorno de ejecución subyacente.
- **Ausencia de estado**
- **Capacidad de ser descubiertos:** complementados con metadatos que hacen que puedan ser adecuadamente descubiertos e interpretados.
- **Capacidad de ser combinados:** formando lógicas más complejas.

No conserva información entre una invocación y la siguiente

- Escalabilidad horizontal: se pueden ejecutar en múltiples nodos
- Confiabilidad y tolerancia a fallos: Si un nodo cae, otro puede continuar
- Desacoplamiento

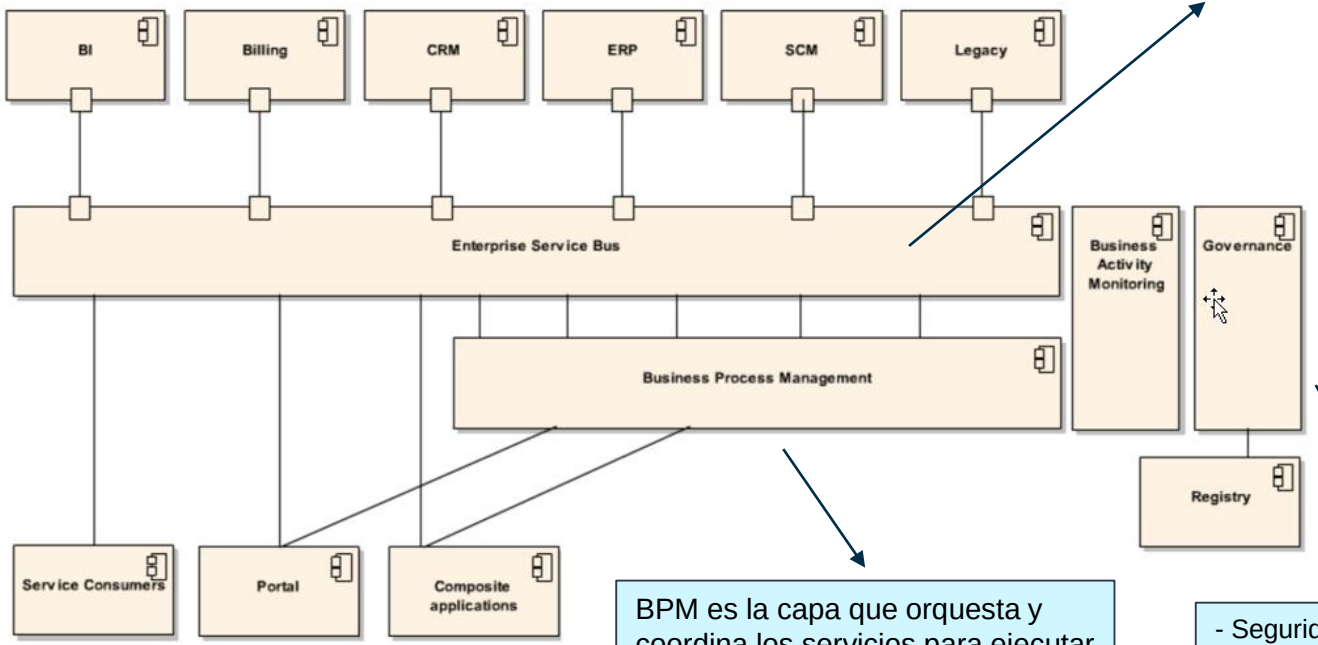
Arquitectura con integraciones punto a punto

- Muchos sistemas conectados entre sí con integraciones individuales
- Cada nuevo sistema obliga a tocar medio mapa



Arquitectura con integración SOA

- Middleware que actúa como un canal de comunicación central entre servicios en una arquitectura SOA
- Permite que distintos sistemas y servicios se hablen entre sí, sin conectarse directamente.



BPM es la capa que orquesta y coordina los servicios para ejecutar los procesos de negocio de punta a punta

- Seguridad
- Políticas de acceso
- Control de versiones
- Monitoreo y auditoría
- Gestión de tráfico
- Cumplimiento de contratos

CASO TELECOM – IBM:
Súper Integración

- Integrar aplicaciones de manera más ordenada y centralizada
- Puede volverse pesada: requiere gobierno, control de contratos, versionado y monitoreo

SOA - Conclusiones

- SOA es aun hoy el modelo mas usado en las empresas para integrar sus aplicaciones corporativas
- Resuelve problemas clásicos de integración en forma satisfactoria
- Necesita un governance centralizado, apto para ciertas organizaciones y contexto (no para todos)
- Presenta limitaciones en cuanto a escalabilidad, fault tolerance, desarrollo rápido y deployments frecuentes

Microservicios

Desafío actual: entregar cambios rápido, pero sin destruir producción

Software hoy: moverse rapido sin romper nada

Software delivery performance metric	Elite	High	Medium	Low
Deployment frequency For the primary application or service you work on, how often does your organization deploy code to production or release it to end users?	On-demand (multiple deploys per day)	Between once per week and once per month	Between once per month and once every 6 months	Fewer than once per six months
Lead time for changes For the primary application or service you work on, what is your lead time for changes (i.e., how long does it take to go from code committed to code successfully running in production)?	Less than one hour	Between one day and one week	Between one month and six months	More than six months
Time to restore service For the primary application or service you work on, how long does it generally take to restore service when a service incident or a defect that impacts users occurs (e.g., unplanned outage or service impairment)?	Less than one hour	Less than one day	Between one day and one week	More than six months
Change failure rate For the primary application or service you work on, what percentage of changes to production or released to users result in degraded service (e.g., lead to service impairment or service outage) and subsequently require remediation (e.g., require a hotfix, rollback, fix forward, patch)?	0%-15%	16%-30%	16%-30%	16%-30%

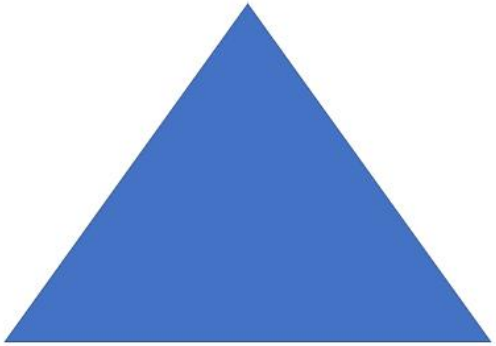
CI/CD, tests automáticos, despliegues y feature flags

Que se necesita?

Prácticas

Procesos

DevOps - Continuous delivery/deployment



Organización

Equipos pequeños y autónomos

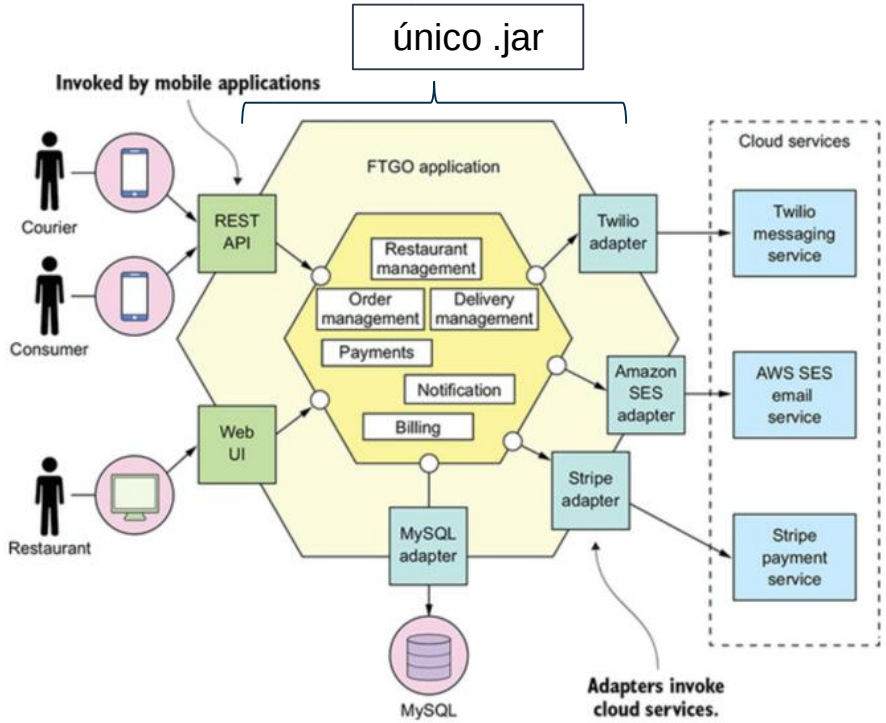
Agilidad

Arquitectura

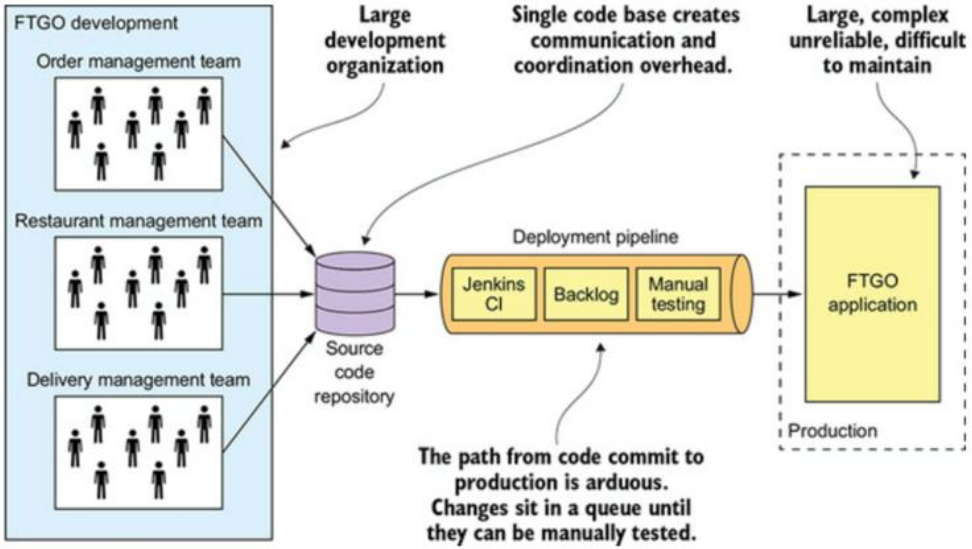
?

No es suficiente

Enfoque tradicional: arquitectura monolítica

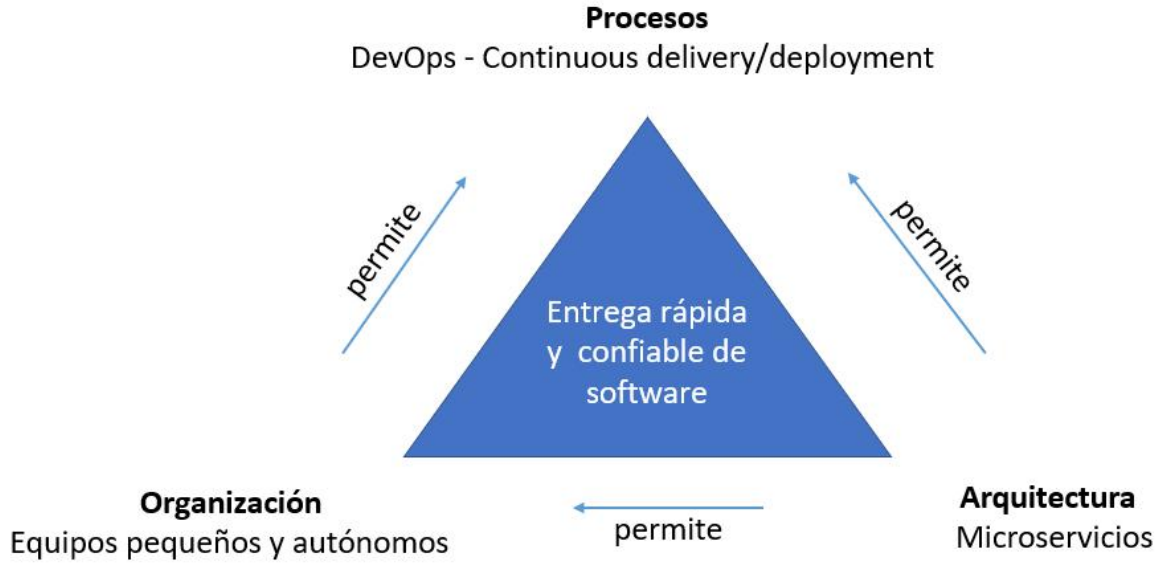


Problema: con el tiempo, es una limitación



- Código complejo
- Lento pasaje de commit a deploy
- Dificultades para escalar
- Problemas de fault tolerance
- Cautivos en un stack tecnológico potencialmente obsoleto

Microservicios: habilitan moverse rápido sin romper nada



No todas son ventajas

- No es fácil definir correctamente los servicios.
- Multiples bases datos: como se coordina la transaccionalidad?
- Los microservicios constituyen un sistema distribuido, que tiene ciertas complejidades de desarrollo, testeo y deployment.
- El deployment de features que impactan multiples servicios debe ser coordinado cuidadosamente

- *Se debe decidir prudentemente cuando utilizar microservicios, porque existen* ***complejidades de implementación***

Entonces… ¿Cuándo no usar microservicios?

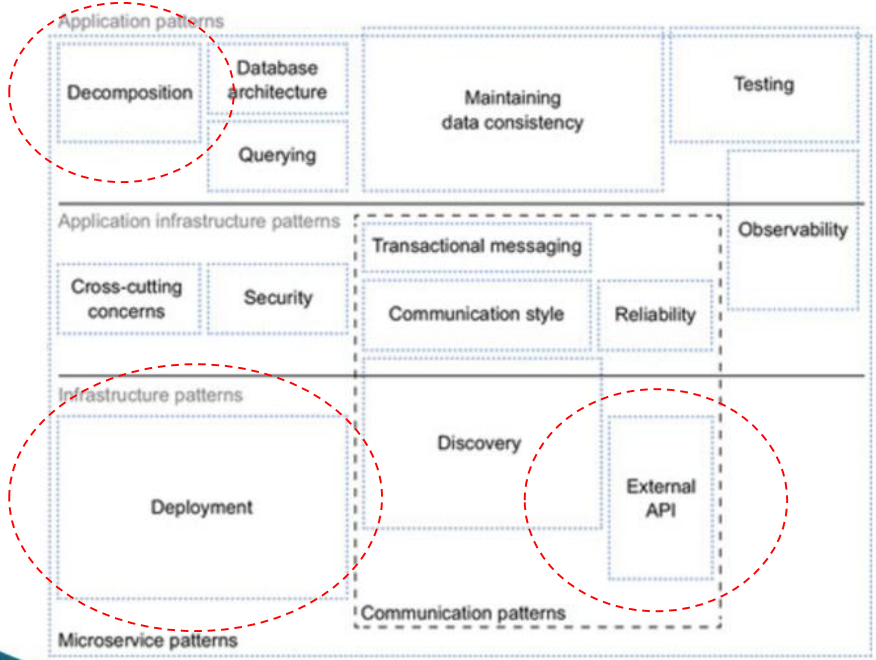
Microservicios != Magia

- Agregan complejidad distribuida
- Problemas de datos
- Trazabilidad
- Latencia
- Coordinación

No conviene usarlos si:

- El dominio es simple
- El equipo es chico
- No hay madurez DevOps
- No se necesita escalar partes por separado

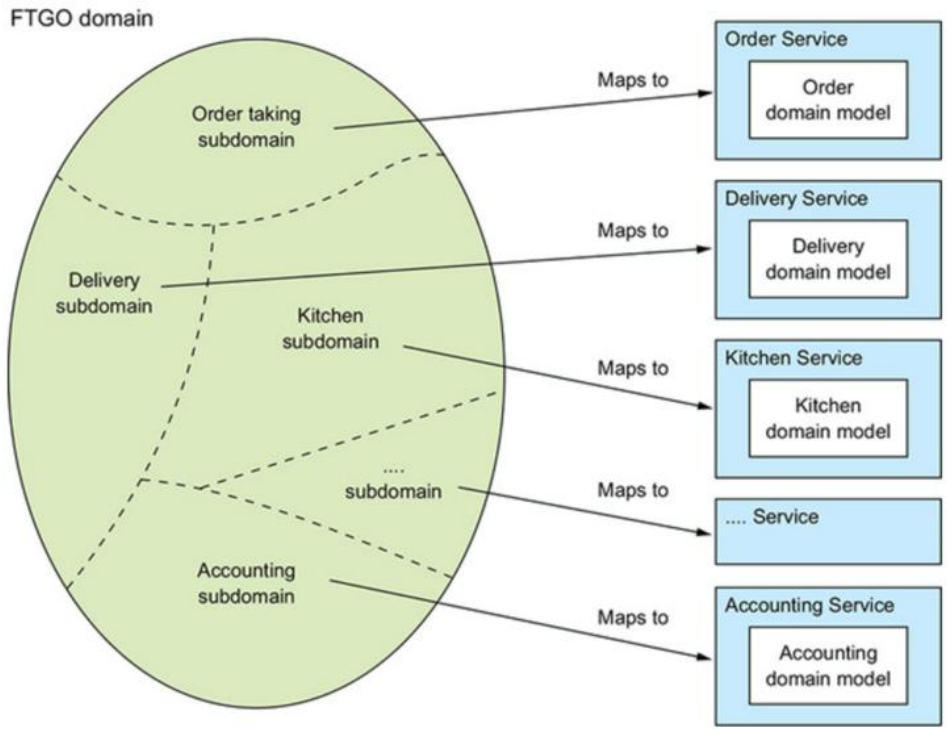
Microservices patterns



Conjunto de patterns que ayudan a la resolucion de problemas típicos de arquitectura diseño, desarrollo y operación en arquitectura de microservicios

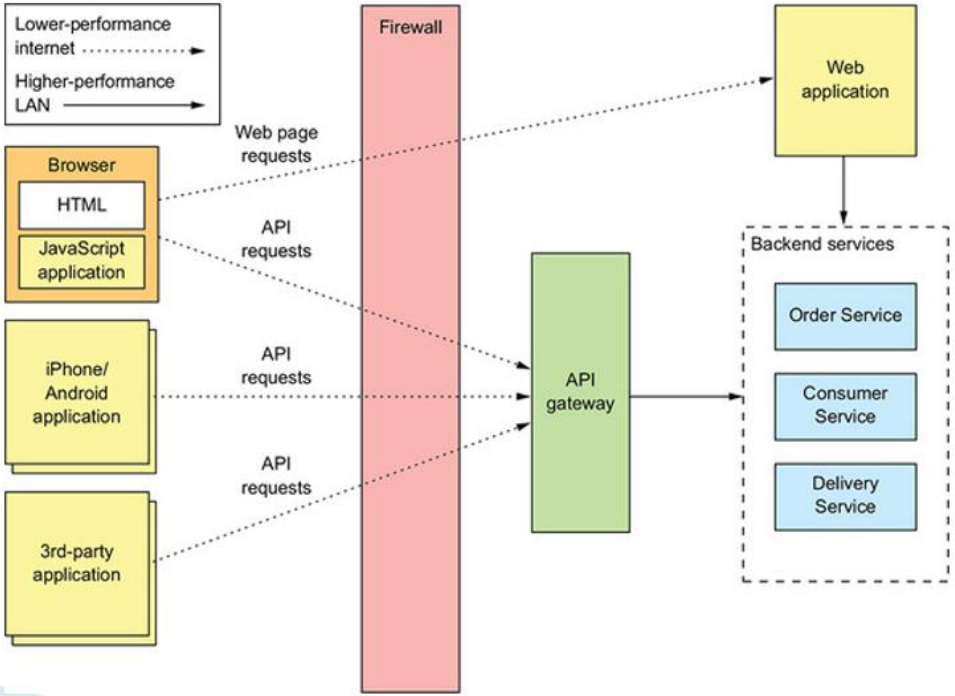
- Domain-Driven Design: No se separa por capas técnicas, sino por capacidades del negocio.
- Ejemplo, en banca digital: accounts, transfers, loans, cards, fraud-detection, notifications

Decomposition by subdomain



- Concentra el acceso externo,
- Enruta requests a servicios internos
- Aplica autenticación, logging, …

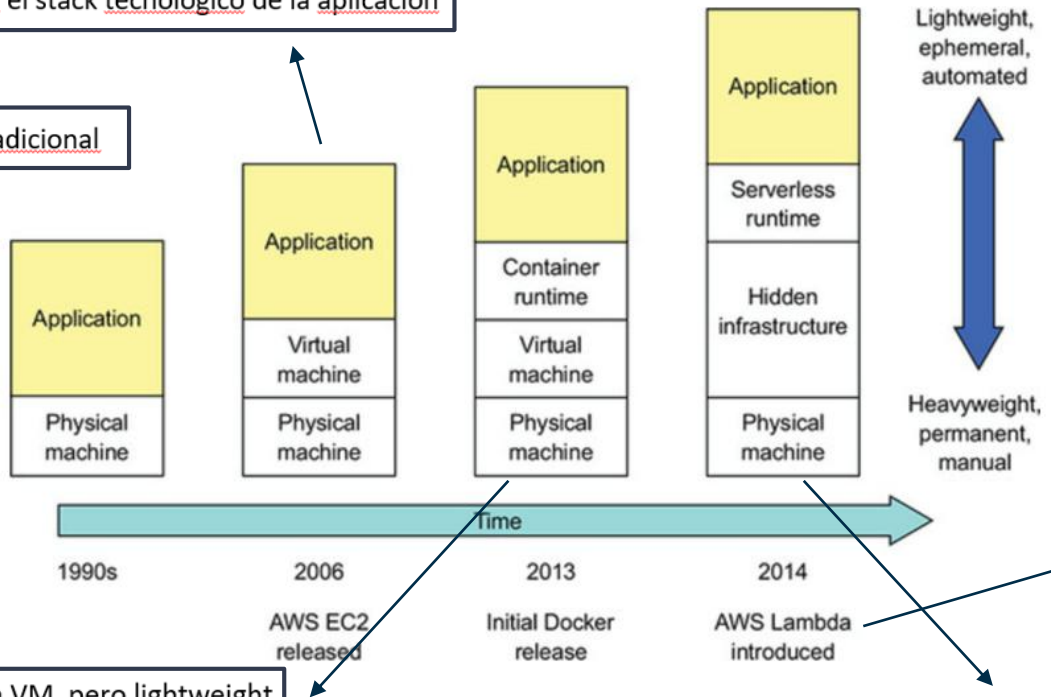
Communication Patterns: API gateway



Deployment: como evolucionó la infraestructura

Servicio en una VM: encapsula el stack tecnológico de la aplicación

Formato específico del lenguaje: tradicional

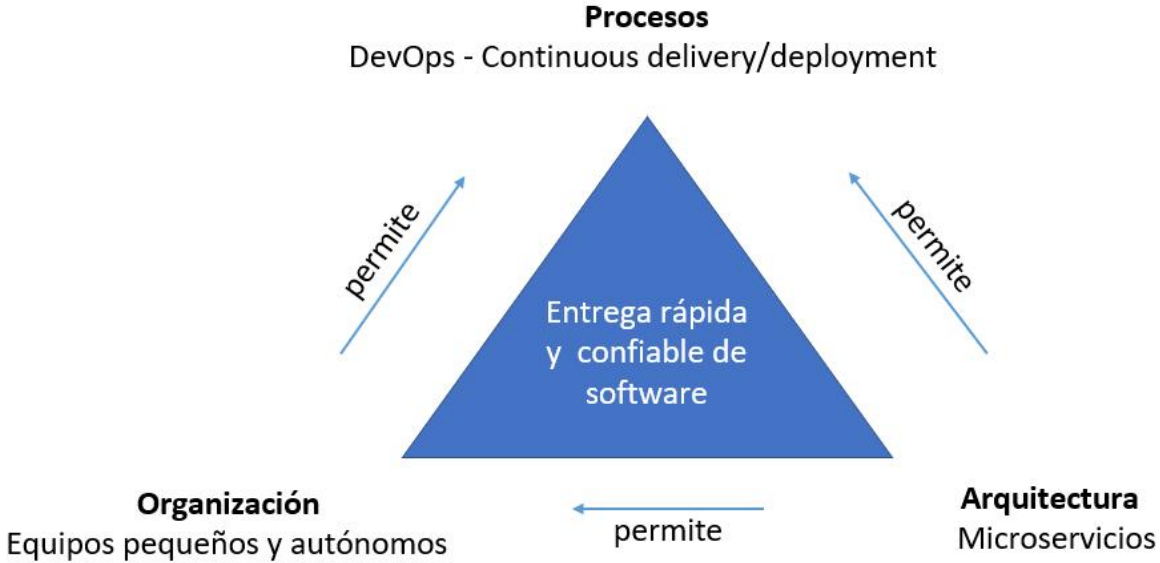


Modelo *serverless* de Amazon

Servicio en un container: similar a VM, pero lightweight

Serverless: desaparece el concepto de maquina, sea virtual o en container

Conclusión: los microservicios permiten entrega rápida y confiable de software en los contextos adecuados



Microservicios → entrega rápida y confiable → solo si:
- DevOps
- Equipos autónomos
- Buena arquitectura